TMDB Movie Analytics Lakehouse

Final Architectural & FinOps Report

Report date	October 26, 2026
Environment	Databricks workspace dbc-4532e676-0f08 (Workspace ID 7474657753678173)
Unity Catalog namespace	`workspace.tmdb`
Compute	Serverless: all-purpose (notebooks), jobs (scheduled job tasks), SQL warehouse (BI / SQL editor); see Section 6.1
Project root	`/Users/l242519@lhr.nu.edu.pk/TMDB-Data-Engineering-Project`
BI tool	Power BI Desktop (DirectQuery to the Databricks SQL warehouse)
Data verification	All counts, audit-log content, schemas and billing figures in this report were queried live on 2026-10-26 (audit table, row counts, information_schema, system.billing, system.lakeflow). Bronze physical properties come from DESCRIBE DETAIL. The Spark UI profile in Section 5.4 was captured on 2026-10-25.

Contents

- 1. Executive summary and medallion architecture
- 2. Decision log: technical rationale and architecture defense
- 3. Orchestration and audit-logging evaluation
- 4. Business intelligence layer (Power BI)
- 5. Spark UI and DAG analysis (Gold fact-table MERGE)
- 6. FinOps evaluation and scaling analysis
- 7. Verification basis
- 8. Deliverables map and conclusion

1. Executive Summary & Medallion Lakehouse Architecture

1.1 High-level overview

The TMDB Movie Analytics Lakehouse is a batch-oriented data platform that ingests raw movie metadata from The Movie Database (TMDB) REST API and refines it step by step into an analytics-ready Gold star schema, which a Power BI dashboard consumes. It follows the **Medallion Architecture**, the standard Databricks pattern of three layers of increasing data quality (Bronze, Silver, Gold). Every table is a Delta Lake table governed by Unity Catalog under the `workspace.tmdb` namespace. The business purpose is to analyze global movie trends, ratings, production budgets, revenues and genre performance over time.

Key architectural characteristics, each tied to evidence gathered on 2026-10-26:

- **Immutable raw layer.** The full, unmodified TMDB JSON payload is stored verbatim in Bronze. Bronze currently holds 989,664 rows across 5 ingestion batches, so any parsing change can be replayed without calling the API again.
- **Typed Silver with a quarantine safety net.** Silver applies explicit types and cleaning rules. Malformed records are routed to `quarantine_movies` instead of failing the run. The quarantine table holds 0 rows.
- **MERGE-based, re-runnable loads.** Silver and Gold are loaded with Delta MERGE keyed on the business keys. Across 10 successful Silver and Gold runs, Silver stayed at exactly 197,871 rows (equal to the number of distinct movie IDs in Bronze) and Gold row counts did not change (Section 2.2).
- **Dimensional Gold layer.** `dim_movies`, `dim_date` and `fact_movie_performance`, with Liquid Clustering (CLUSTER BY) for physical layout. In Unity Catalog the genre data lives in the Silver tables `silver_genres` and `silver_genre_bridge`; the Power BI model exposes it as a `dim_genre` table (Section 4.2).
- **Chained orchestration.** Bronze, Silver and Gold are chained by a master notebook or as dependent tasks of a Databricks Job, so a new file flows to Gold without manual steps.
- **End-to-end auditability.** Every run, successful or failed, is logged to `workspace.tmdb.pipeline_execution_logs` (25 rows at the 2026-10-26 snapshot, 10 of them captured failures). The audit table is also visualized in the BI dashboard.
- **Low-cost serverless compute.** A full Gold refresh (403,857 row-level operations across three tables) took between 39.5 s and 58.5 s over five logged runs (mean 46.3 s).

1.2 Architecture diagram

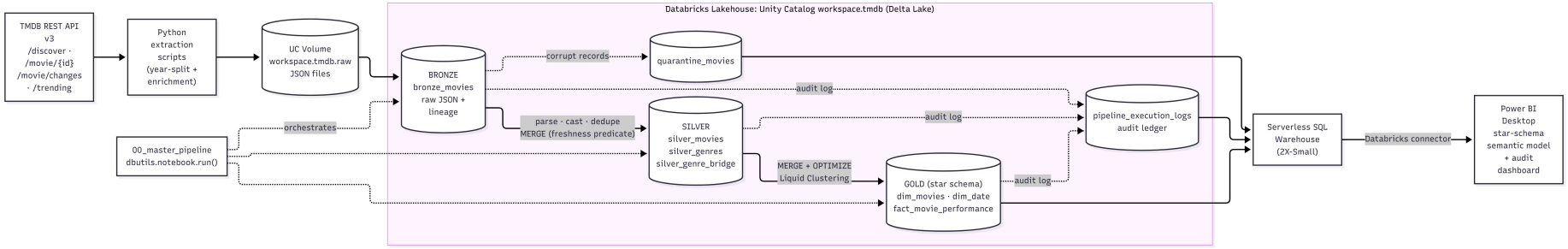


Figure 1.1: End-to-end data flow from the TMDB API through Bronze, Silver and Gold into Power BI. Solid arrows are data flow; dotted arrows are control, audit and exception paths.

Reading the diagram, left to right

Step	Component	What it does
1	TMDB REST API v3 and Python extraction scripts	Year-split discovery plus enrichment (/discover, /movie/{id}, /movie/changes, /trending) writes raw JSON files.
2	UC Volume `workspace.tmdb.raw`	Landing area for the JSON files before Bronze ingestion.
3	Bronze: `bronze_movies`	Raw JSON plus lineage columns. Corrupt records are diverted to `quarantine_movies`.
4	Silver: `silver_movies`, `silver_genres`, `silver_genre_bridge`	Parse, cast and deduplicate with MERGE (freshness predicate).
5	Gold: `dim_movies`, `dim_date`, `fact_movie_performance`	MERGE plus OPTIMIZE; Liquid Clustering.
6	`00_master_pipeline`	Orchestrates the three stages with dbutils.notebook.run() (dotted control arrows).
7	`pipeline_execution_logs`	Audit ledger written by every stage (dotted audit-log arrows).
8	Serverless SQL warehouse (2X-Small) and Power BI Desktop	Gold, the audit table and quarantine table are read through the Databricks connector.

1.3 Verified table inventory (queried 2026-10-26)

Table	Layer	Row count	Notes
bronze_movies	Bronze	989,664	5 batches: 197,520 + 4 × 198,036. Unpartitioned, unclustered. Detail: 7 files, 212,440,218 bytes (about 202.6 MiB).
silver_movies	Silver	197,871	24 columns. Equals the count of distinct movie IDs across all Bronze batches.
silver_genres	Silver (reference)	19	Genre lookup (genre_id, genre_name, lineage columns).
silver_genre_bridge	Silver (bridge)	16,760	Movie-to-genre many-to-many bridge.
dim_movies	Gold	197,871	CLUSTER BY (movie_id)
dim_date	Gold	8,142	CLUSTER BY (date_key)
fact_movie_performance	Gold	197,844	CLUSTER BY (movie_id, date_key)
quarantine_movies	Quarantine	0	Rejected-record holding area.
pipeline_execution_logs	Audit	25	Run-level execution ledger.

Raw API payloads are first staged as JSON files in the `workspace.tmdb.raw` Unity Catalog Volume and then loaded into `bronze_movies`. Genre data is held in the Silver tables `silver_genres` and `silver_genre_bridge`; the Power BI model exposes it as `dim_genre` (Section 4.2).

1.4 Bronze layer: raw JSON ingestion

Bronze is the immutable landing zone for raw TMDB responses and follows a schema-on-read approach. The source JSON is stored verbatim as a string and no parsing, typing or business rules are applied. Ingestion supports a full load (year-split discovery queries to stay under TMDB's 500-page limit, enriched with budget, revenue, runtime and keywords) and an incremental load (daily changed IDs from /movie/changes and /trending/movie/day).

Column	Type	Role
raw_record	STRING	Full, unmodified TMDB JSON payload
batch_id	STRING	Ingestion batch identifier, e.g. 20261009_153323
source_endpoint	STRING	TMDB endpoint the record came from
source_collection	STRING	Origin: full_load, trending or changed
source_file	STRING	Source file lineage
pulled_at	STRING	API pull date (freshness tie-breaker for Silver). Holds 2 distinct values: 2026-10-01 and 2026-10-08
load_timestamp	TIMESTAMP	Delta load time (audit lineage)

Physical state (DESCRIBE DETAIL): format Delta, no partition columns, no clustering columns, 7 data files, 212,440,218 bytes, last modified 2026-10-25 13:19:55 UTC. Deletion vectors and row tracking are enabled; zstd Parquet compression is used.

Verified ingestion history. Five successful Raw-to-Bronze runs were logged (2026-10-09, 10-11, 10-23 twice, 10-25). The first loaded 197,520 rows; each of the other four appended a 198,036-row snapshot. Total: 197,520 + 4 × 198,036 = 989,664 rows across batches 20261009_153323, 20261011_090006, 20261023_160110, 20261023_160756 and 20261025_131923. Of 197,871 distinct movie IDs, all 197,871 appear in more than one batch. Bronze therefore behaves as an append-only history of full snapshots, and Silver is where the data is reduced to one row per movie.

1.5 Silver layer: typed, cleaned, enforced

Silver parses the raw JSON with an explicit PySpark StructType (inferSchema is disabled on every ingestion path) into a typed relational model. The genres array is exploded into `silver_genre_bridge` (keyed on movie_id, genre_id) and `silver_genres` (keyed on genre_id).

Column	Type	Column	Type
movie_id (PK)	INT	runtime	INT
title	STRING	status	STRING
original_title	STRING	tagline	STRING
overview	STRING	source_collection	STRING
release_date	DATE	source_file	STRING
popularity	DOUBLE	pulled_at	DATE
vote_average	DOUBLE	batch_id	STRING
vote_count	BIGINT	load_timestamp	TIMESTAMP
budget	BIGINT	release_year (derived)	INT
revenue	BIGINT	adult	BOOLEAN
original_language	STRING	backdrop_path	STRING
updated_at	TIMESTAMP	poster_path	STRING

Cleaning rules. Fields are cast to explicit Spark types, whitespace is trimmed, null IDs and blank titles are filtered, release_date is cast with try_to_date (yyyy-MM-dd), release_year is derived from it, original_language is normalized with TRIM and UPPER, and lineage metadata is carried forward from Bronze.

Schema enforcement: PERMISSIVE mode with quarantine. Malformed payloads do not halt the pipeline. They are routed to `workspace.tmdb.quarantine_movies` (layer, batch_id, source_file, reject_reason, raw_record, load_timestamp). Current quarantine count: 0.

MERGE-based deduplication. Silver is loaded with a Delta MERGE INTO on movie_id. A matched row is updated when the incoming pull is newer:

```
MERGE INTO workspace.tmdb.silver_movies AS t
USING silver_source_view AS s
ON t.movie_id = s.movie_id
WHEN MATCHED AND s.pulled_at > t.pulled_at THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Silver's pulled_at is a DATE, so freshness is compared at day granularity (the table holds two distinct values, 2026-10-01 and 2026-10-08). Section 2.2 shows the audit-log record of repeated loads.

1.6 Gold layer: star schema

The Gold layer is a Kimball-style star schema built for analytical consumption. Descriptive attributes live in dimensions, a date dimension is generated from distinct release dates, and the fact table holds numeric measures plus the derived metrics profit and roi_percentage.

Table	Type	Role
fact_movie_performance	Fact	Budget, revenue, profit, ROI, popularity, vote average, vote count
dim_movies	Dimension	Title, original title, overview, release date, status, tagline
dim_date	Dimension	Year, quarter, month, day, day of week
silver_genres / silver_genre_bridge	Reference + bridge (Silver)	Genre names and the movie-to-genre many-to-many relationship (19 genres, 16,760 pairs)
dim_genre (Power BI model)	Dimension	Columns genre_name and movie_id; exposes genre data to the semantic model

Gold data dictionary

dim_movies **(7 cols)**	Type	dim_date **(7 cols)**	Type
movie_id (PK)	BIGINT	date_key (PK)	INT (YYYYMMDD)
title	STRING	release_date	DATE

dim_movies (7 cols)	Type	dim_date (7 cols)	Type
original_title	STRING	year	INT
overview	STRING	quarter	INT (1–4)
release_date	DATE	month	INT (1–12)
status	STRING	day	INT
tagline	STRING	day_of_week	INT (1 = Sunday … 7 = Saturday)

fact_movie_performance (11 cols)	Type	Notes
movie_id (FK → dim_movies)	BIGINT	Part of the grain key
date_key (FK → dim_date)	INT	Part of the grain key
budget	BIGINT	Production budget (USD)
revenue	BIGINT	Global box-office revenue (USD)
profit	BIGINT	Derived: revenue − budget
roi_percentage	DOUBLE	Derived: (revenue − budget) / budget × 100; NULL when budget ≤ 0
popularity	DOUBLE	TMDB popularity score
vote_average	DOUBLE	User rating (0–10)
vote_count	BIGINT	Number of user votes
batch_id	STRING	Lineage
load_timestamp	TIMESTAMP	Lineage

Key integrity properties

- dim_movies.movie_id is the primary key: 197,871 rows, equal to the 197,871 distinct movie IDs in Bronze and to the Silver row count.
- dim_date.date_key (integer YYYYMMDD) is the primary key: 8,142 rows. The fact table's date_key references it.
- The fact grain is one row per movie per release date, with composite key (movie_id, date_key).
- 27 Silver rows with a NULL release_date are excluded from the fact table because they cannot join to dim_date: 197,871 − 197,844 = 27.
- A movie with several genres appears once per genre in the genre structure (16,760 movie-genre pairs in silver_genre_bridge), which is why genre analysis is kept out of the fact grain.

Read optimization: Liquid Clustering

```
CREATE TABLE IF NOT EXISTS workspace.tmdb.dim_movies (
  movie_id BIGINT, title STRING, original_title STRING,
  overview STRING, release_date DATE, status STRING, tagline STRING
) CLUSTER BY (movie_id);

CREATE TABLE IF NOT EXISTS workspace.tmdb.dim_date (
  date_key INT, release_date DATE, year INT, quarter INT,
  month INT, day INT, day_of_week INT
) CLUSTER BY (date_key);

CREATE TABLE IF NOT EXISTS workspace.tmdb.fact_movie_performance (
  movie_id BIGINT, date_key INT, budget BIGINT, revenue BIGINT,
  profit BIGINT, roi_percentage DOUBLE, popularity DOUBLE,
  vote_average DOUBLE, vote_count BIGINT, batch_id STRING,
  load_timestamp TIMESTAMP
) CLUSTER BY (movie_id, date_key);
```

Gold table	Clustering key(s)	Why
dim_movies	movie_id	Primary-key lookups cluster to a narrow file range
dim_date	date_key	Time-range queries prune to the relevant dates
fact_movie_performance	movie_id, date_key	Matches the MERGE join keys and the grain key, so upserts and analytical filters both skip irrelevant files

OPTIMIZE runs on `dim_movies` and `fact_movie_performance` after each Gold load.

Validation (live, 2026-10-26)

- Row counts: dim_movies 197,871 · dim_date 8,142 · fact_movie_performance 197,844 · quarantine_movies 0.
- Audit summary: Raw-to-Bronze 5 runs · Bronze-to-Silver 15 runs (5 success, 10 failure) · Silver-to-Gold 5 runs · 0 rows quarantined.
- Star-schema top-10 revenue join verified earlier in the project (Avatar leads at $2.92B revenue, 1133.63% ROI).

2. Decision Log: Technical Rationale & Architecture Defense

2.1 Schema drift and corrupt records: PERMISSIVE parsing + quarantine over strict halt

Parsing an external API in FAILFAST mode is an availability liability: one malformed record or one unexpected type change can crash the whole batch, block every downstream layer and need an engineer before the next run. The PERMISSIVE + quarantine pattern treats parsing as a filter, not a gate:

- **Good records keep flowing,** so latency is unaffected by a handful of bad records.
- **Bad records are captured, not lost.** They land in `quarantine_movies` with a reject_reason and the full raw payload for root-cause analysis and replay.
- **Drift is observable, not fatal.** Because Bronze keeps the raw JSON, new source fields can be adopted later by extending the Silver StructType and replaying, with no API re-pull.
- **The blast radius is contained.** A quarantined record never enters Silver and so can never reach Gold.

Results to date. No source-side schema drift has occurred: `quarantine_movies` holds 0 rows and every payload parsed cleanly. The 10 failures in the audit log are development-time DELTA_MERGE_UNRESOLVED_EXPRESSION errors on 2026-10-11, where a MERGE clause referenced columns that were not resolvable. Their exact breakdown from the logged error messages:

Count	Target	Clause	Unresolved column(s)
7	`silver_movies`	UPDATE	budget, release_year, adult, backdrop_path, poster_path, original_language, updated_at (one per failed run)
1	`silver_movies`	INSERT	budget
2	`silver_genres`	UPDATE	title (the movie-column mapping was applied to the genre table, whose columns are genre_id, genre_name, batch_id, load_timestamp, updated_at)

They were fixed by giving every column an explicit source-to-target mapping, and the 09:31:24 run then succeeded. Each failure ended within about 15 seconds (eight within 1.6 s).

2.2 Idempotency strategy: key-based MERGE and what the audit log shows

Idempotency (re-running a load with the same input yields the same table state) is the most important reliability property of a batch pipeline, because duplicate deliveries and retried runs are routine. Silver uses MERGE on movie_id and Gold uses key-based MERGE (whenMatchedUpdateAll / whenNotMatchedInsertAll), so a key match updates in place and a key miss inserts once.

Evidence (audit log + live counts)	Observed result
2026-10-09 15:34: first Silver load of batch 20261009_153323	197,519 inserted, 0 updated
2026-10-09 16:00: the same batch processed again	0 inserted, 197,519 updated
Bronze-to-Silver (delta_parse) on 10-11, 10-23, 10-25 over a Bronze table that grew to 989,664 rows	Each run logged 197,871 inserted, 0 updated. Silver stayed at 197,871 rows, equal to Bronze's 197,871 distinct movie IDs
Silver-to-Gold, 5 runs (10-21 to 10-25)	Every run logged 0 inserted, 403,857 updated (197,871 + 8,142 + 197,844). Gold counts were unchanged after every run

Conclusion supported by the data: repeated runs, including five full Bronze snapshots that each re-delivered the same 197,871 movies, never produced duplicate rows in Silver or Gold, so the pipeline converges to the same table state.

2.3 Physical layout: Liquid Clustering vs. Hive-style partitioning

Why not partition columns? Hive-style partitioning splits a table into directory trees. On this workload that is a poor fit for three reasons:

- **Small-file explosion.** About 200K movies over 8,142 distinct release dates would create thousands of directories, each holding a few small files. That slows Delta snapshot listing, raises storage request costs and slows every OPTIMIZE and VACUUM.
- **Cardinality mismatch.** Genre is a many-to-many attribute (a movie has several genres through the bridge), so it cannot be a partition column on movie-grain tables without denormalization and would still skew.
- **Rigid layout.** Partition keys cannot be changed without a full rewrite.

Why Liquid Clustering? It replaces directory partitioning with incremental, file-level data layout:

- **No directory explosion.** Data is laid out in clustered files rather than directory trees.
- **Layout on the query keys.** CLUSTER BY on movie_id, date_key and (movie_id, date_key) co-locates rows with similar key values, so point lookups and date-range queries can skip irrelevant files as tables grow.
- **Incremental reclustering.** OPTIMIZE (run after each Gold MERGE) reclusters incrementally, with no full-rewrite cost.
- **Evolution-friendly.** Clustering keys can be changed later with ALTER TABLE ... CLUSTER BY.

Column choices. movie_id (about 197K distinct values) is the primary key and MERGE join key; date_key is the time-slicing key for trend queries; the fact table uses both because they form its grain key and match the MERGE predicate. High-cardinality keys that would be disastrous as Hive partitions work well as clustering keys. Bronze and Silver are intentionally unclustered: the pipeline reads and writes them in full, so clustering would add maintenance cost with no read benefit.

2.4 PII and data governance

- **Classification.** The dataset is public movie metadata (titles, overviews, dates, budgets, revenues, ratings, genres, image paths). No Silver or Gold column holds personal data. The adult column is a TMDB content flag on the movie, not a personal attribute.
- **Conclusion.** No PII masking or tokenization was required, and none was applied.
- **Credentials.** The TMDB API key is supplied to the extraction script at run time and is not stored in the repository; credential files are excluded through .gitignore. Power BI authenticates with a personal access token that stays on the author's machine.
- **If PII were introduced** (for example user-level reviews), the design supports pseudonymizing identifiers in Silver, Unity Catalog column masks and row filters on Gold, SELECT grants on Gold only, and lineage and audit logs to trace access.
- **Access model.** All tables live in Unity Catalog, so access is governed per table with GRANT and REVOKE. Bronze is never exposed to BI.

3. Orchestration & Audit Logging Evaluation

3.1 Master orchestration: 00_master_pipeline

The `00_master_pipeline` notebook runs the three stages sequentially, each as an isolated child-notebook run, so a new file processed into Silver refreshes Gold in the same execution.

Order	Stage	Child notebook
1	Raw-to-Bronze	`bronze/bronze_injestion`
2	Bronze-to-Silver	`silver/silver_injestion`
3	Silver-to-Gold	`gold/gold_transformation`

```
dbutils.widgets.text("load_type", "incremental", "Load type: 'full' or 'incremental'")
dbutils.widgets.text("batch_id", "", "Batch ID (blank = process all batches)")
load_type = dbutils.widgets.get("load_type").strip().lower()
batch_id = dbutils.widgets.get("batch_id").strip()
if load_type not in ("full", "incremental"):
    raise ValueError(f"Invalid load_type '{load_type}'. Use 'full' or 'incremental'.")
PIPELINE_STAGES = [
    ("Raw-to-Bronze", "bronze/bronze_injestion"),
    ("Bronze-to-Silver", "silver/silver_injestion"),
    ("Silver-to-Gold", "gold/gold_transformation"),
]
STAGE_TIMEOUT_SECONDS = 3600
try:
    for stage_name, stage_path in PIPELINE_STAGES:
        dbutils.notebook.run(stage_path, STAGE_TIMEOUT_SECONDS,
                             {"load_type": load_type, "batch_id": batch_id})
        stage_results.append({"stage": stage_name, "status": "SUCCESS", ...})
except Exception as e:
    print(f"MASTER PIPELINE FAILED at stage '{current_stage}' after {elapsed}s")
    raise
```

- **Parameters.** load_type (default incremental) selects full vs. incremental semantics; batch_id (default blank) restricts processing to one batch. An invalid load_type raises a ValueError before any data moves.
- **Per-stage timeout.** dbutils.notebook.run() with a 3,600-second timeout stops a hung stage from wedging the orchestrator.
- **Failure isolation.** The try/except records which stage failed, prints a per-stage summary and re-raises so the run shows as FAILED. Completed stages keep their results.
- **Dependency order.** Bronze completes before Silver, and Silver before Gold, matching the data dependencies.

Scheduled execution (Databricks Job). The same chain is deployed as a serverless Databricks Job with three dependent notebook tasks (raw_to_bronze → bronze_to_silver → silver_to_gold), each receiving load_type = incremental and with retries disabled so a failure writes exactly one FAILURE row to the audit table. The job is split into tasks instead of calling `00_master_pipeline` because the workspace allows one concurrent active run, and on serverless compute each dbutils.notebook.run() starts a separate run that the quota rejects with REQUEST_LIMIT_EXCEEDED.

Job runs recorded in system.lakeflow.job_run_timeline

Job ID	Run ID	Start (UTC)	End (UTC)	Result	Trigger
958812722870319	725911842299222	2026-10-21 10:33:47	2026-10-21 10:34:20	SUCCEEDED	ONETIME
897910413932812	360478832524376	2026-10-23 15:19:14	2026-10-23 15:20:17	TIMED_OUT	ONETIME
882712524812831	358653466929734	2026-10-25 12:38:36	2026-10-25 12:59:08	ERROR	ONETIME
882712524812831	700510410216817	2026-10-25 13:18:42	2026-10-25 13:24:00	ERROR	ONETIME

Run 700510410216817 (13:18:42 to 13:24:00 on 10-25) encloses audit rows 22 to 25, in which every stage logged SUCCESS and Gold executed twice (13:21:37 and 13:23:05).

3.2 Audit logging framework: pipeline_execution_logs

Every stage notebook (including `gold/gold_transformation`) calls the shared `log/00_audit_logging` helper, `log_pipeline_execution(layer, load_type, batch_id, parameter, start_time, end_time, status, rows_inserted,`

`rows_updated, rows_quarantined, error_msg)`, in both its success and exception paths, so failures are first-class events. The table records log_id, layer_processed, load_type, batch_id, parameter_processed, execution start/end time, status, the three row counters and error_message. It holds 25 rows (queried 2026-10-26).

Complete audit log (25 rows, ordered by start time; all times UTC)

#	Layer	load_type	Batch / scope	Start	Status	Ins.	Upd.	Dur.
1	Raw-to-Bronze	full	20261009_153323	10-09 15:33:24	SUCCESS	197,520	0	9.9 s
2	Bronze-to-Silver	full	20261009_153323	10-09 15:34:53	SUCCESS	197,519	0	29.5 s
3	Bronze-to-Silver	full	20261009_153323	10-09 16:00:07	SUCCESS (re-run)	0	197,519	26.2 s
4	Raw-to-Bronze	all	20261011_090006	10-11 09:00:12	SUCCESS	198,036	0	31.3 s
5	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:11:17	FAILURE	0	0	1.6 s
6	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:12:39	FAILURE	0	0	0.7 s
7	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:13:14	FAILURE	0	0	0.7 s
8	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:13:45	FAILURE	0	0	0.7 s
9	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:14:11	FAILURE	0	0	0.6 s
10	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:15:00	FAILURE	0	0	0.7 s
11	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:15:34	FAILURE	0	0	0.7 s
12	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:16:23	FAILURE	0	0	1.0 s
13	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:16:52	FAILURE	0	0	14.4 s
14	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:29:49	FAILURE	0	0	15.1 s
15	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-11 09:31:24	SUCCESS	197,871	0	18.2 s
16	Silver-to-Gold	full	ALL_SILVER_BATCHES	10-21 10:24:08	SUCCESS	0	403,857	41.3 s
17	Raw-to-Bronze	all	20261023_160110	10-23 16:01:15	SUCCESS	198,036	0	28.2 s
18	Raw-to-Bronze	all	20261023_160756	10-23 16:08:00	SUCCESS	198,036	0	8.2 s
19	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-23 16:09:16	SUCCESS	197,871	0	23.8 s
20	Silver-to-Gold	full	ALL_SILVER_BATCHES	10-25 11:20:54	SUCCESS	0	403,857	58.5 s
21	Silver-to-Gold	full	ALL_SILVER_BATCHES	10-25 11:22:57	SUCCESS	0	403,857	39.5 s
22	Raw-to-Bronze	all	20261025_131923	10-25 13:19:23	SUCCESS	198,036	0	33.9 s
23	Bronze-to-Silver	delta_parse	ALL_BRONZE_BATCHES	10-25 13:20:37	SUCCESS	197,871	0	27.9 s
24	Silver-to-Gold	full	ALL_SILVER_BATCHES	10-25 13:21:37	SUCCESS	0	403,857	50.9 s
25	Silver-to-Gold	full	ALL_SILVER_BATCHES	10-25 13:23:05	SUCCESS	0	403,857	41.2 s

Quarantined rows were 0 in every run. The error_message column holds the string 'NONE' (rows 1–3, 16–25) or an empty string (rows 4 and 15) for successful runs, and a full JVM stack trace of 16,878 to 17,792 characters for each of the 10 failures. Durations are computed from the logged start and end timestamps.

3.3 Verified per-layer run metrics

Layer / load type	Status	Runs	Rows inserted	Rows updated	First run	Last run
Raw-to-Bronze / full	SUCCESS	1	197,520	0	10-09	10-09
Raw-to-Bronze / all	SUCCESS	4	792,144	0	10-11	10-25
Bronze-to-Silver / full	SUCCESS	2	197,519	197,519	10-09	10-09
Bronze-to-Silver / delta_parse	SUCCESS	3	593,613	0	10-11	10-25
Bronze-to-Silver / delta_parse	FAILURE	10	0	0	10-11	10-11
Silver-to-Gold / full	SUCCESS	5	0	2,019,285	10-21	10-25

- **Raw-to-Bronze:** 5 successful runs, 989,664 rows, 0 failures logged.
- **Bronze-to-Silver:** 5 successful runs and 10 trapped failures. Current Silver count: 197,871 (deduplicated).
- **Silver-to-Gold:** 5 successful runs, each reconciling exactly: 197,871 + 8,142 + 197,844 = 403,857 row operations (5 × 403,857 = 2,019,285), 0 inserted, 0 quarantined.

The 10 failure rows show the audit framework doing its job. They are historical development failures whose full exception text, layer, batch and timestamps were preserved, which is the forensic record a support team needs: when it broke, what broke and for which batch.

4. Business Intelligence Layer (Power BI)

4.1 Connection and data access

Power BI Desktop connects to the workspace through the Databricks connector using the Server Hostname and HTTP Path of the serverless SQL warehouse, authenticating with a personal access token. The storage mode is DirectQuery, so visuals query the warehouse live. Bronze is never exposed to BI.

Power BI table	Source	Purpose
`fact_movie_performance`	`workspace.tmdb.fact_movie_performance`	Fact: budget, revenue, profit, roi_percentage, popularity, lineage
`dim_movies`	`workspace.tmdb.dim_movies`	Dimension: movie attributes (includes a release_year column in the model)
`dim_date`	`workspace.tmdb.dim_date`	Dimension: date_key, day, day_of_week, month, quarter, release_date, year
`dim_genre`	Genre data (Silver genre tables)	Genre dimension with columns genre_name and movie_id
`pipeline_execution_logs`	`workspace.tmdb.pipeline_execution_logs`	Audit and error-log visuals
`_Measures`	DAX measures table	Holds the Average Rating measure (other measures may be hidden)

4.2 Semantic model

Figure 4.1 is the Power BI Model view. It shows the star schema (dim_date and dim_movies around the fact table), a genre dimension, the measures table and the audit table. The audit table has no relationship to the star, because it describes pipeline operations, not movies.

From	To	Cardinality (as drawn)	Cross-filter (as drawn)
`dim_date[date_key]`	`fact_movie_performance[date_key]`	One-to-many (1:*)	Single (dim → fact)
`dim_movies[movie_id]`	`fact_movie_performance[movie_id]`	One-to-many (1:*)	Single (dim → fact)
`dim_movies[movie_id]`	`dim_genre[movie_id]`	One-to-many (1:*)	Both
`fact_movie_performance`	`dim_genre`	Many-to-many (*:*)	Arrow towards dim_genre

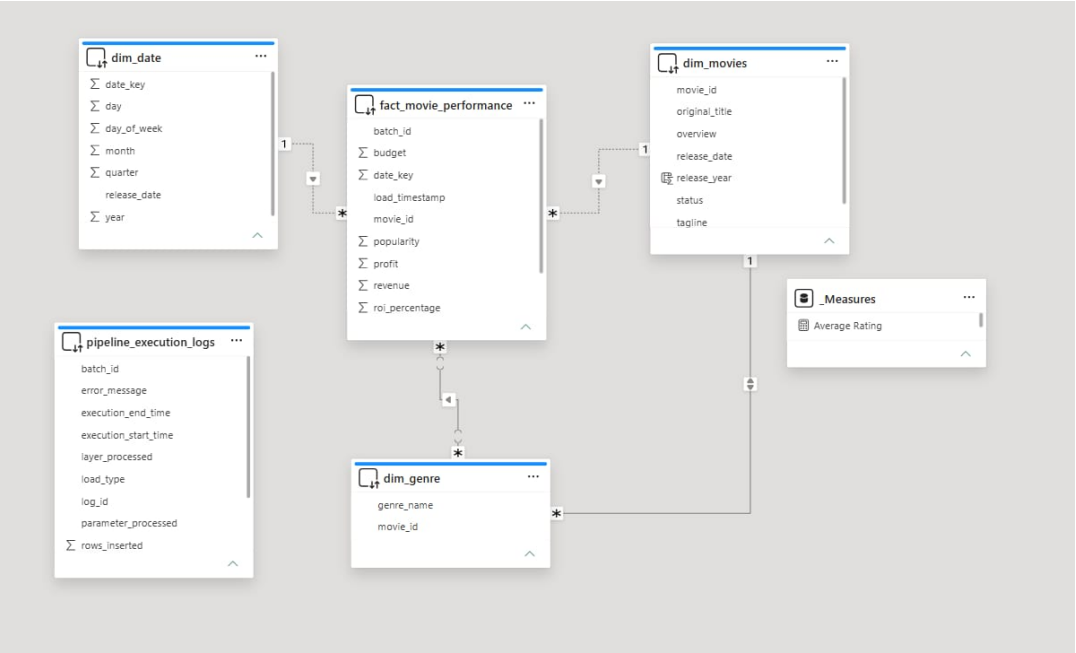


Figure 4.1: Power BI Model view of the semantic model: star schema, genre dimension, measures table and the standalone audit table.

4.3 Business analytics dashboard

The dashboard page is titled *TMDB Movie Performance & Box Office Analytics*. It answers four business questions and offers three kinds of slicer. The values below are those visible in Figure 4.2 at capture time.

#	Business question	Visual	Fields / value shown
Q1	How big is the catalog and how does it perform overall?	Five KPI cards	Average Rating 3.38 · Average ROI 991.08 · Total Movies 198K · Total Revenue 477bn · Total Profit 310bn
Q2	How have revenue and profit trended over time?	Clustered column chart	Total Revenue and Total Profit by release_year; activity concentrates after 2000
Q3	Which genres earn the most revenue?	Horizontal bar chart (sorted)	Total Revenue by genre_name; Adventure, Action and Comedy lead
Q4	Do higher budgets translate to higher revenue?	Scatter / bubble chart	Total Budget (x) vs. Total Revenue (y), bubble size = average popularity, one point per original_title

Slicers: a year range slider (shown at 1931 to 2001 in the capture), a status checklist (In Production, Planned, Post Production, Released, Rumored) and a genre tile selector with 20 tiles, one of which is *Unassigned*. Selections cross-filter the other visuals through the model relationships.

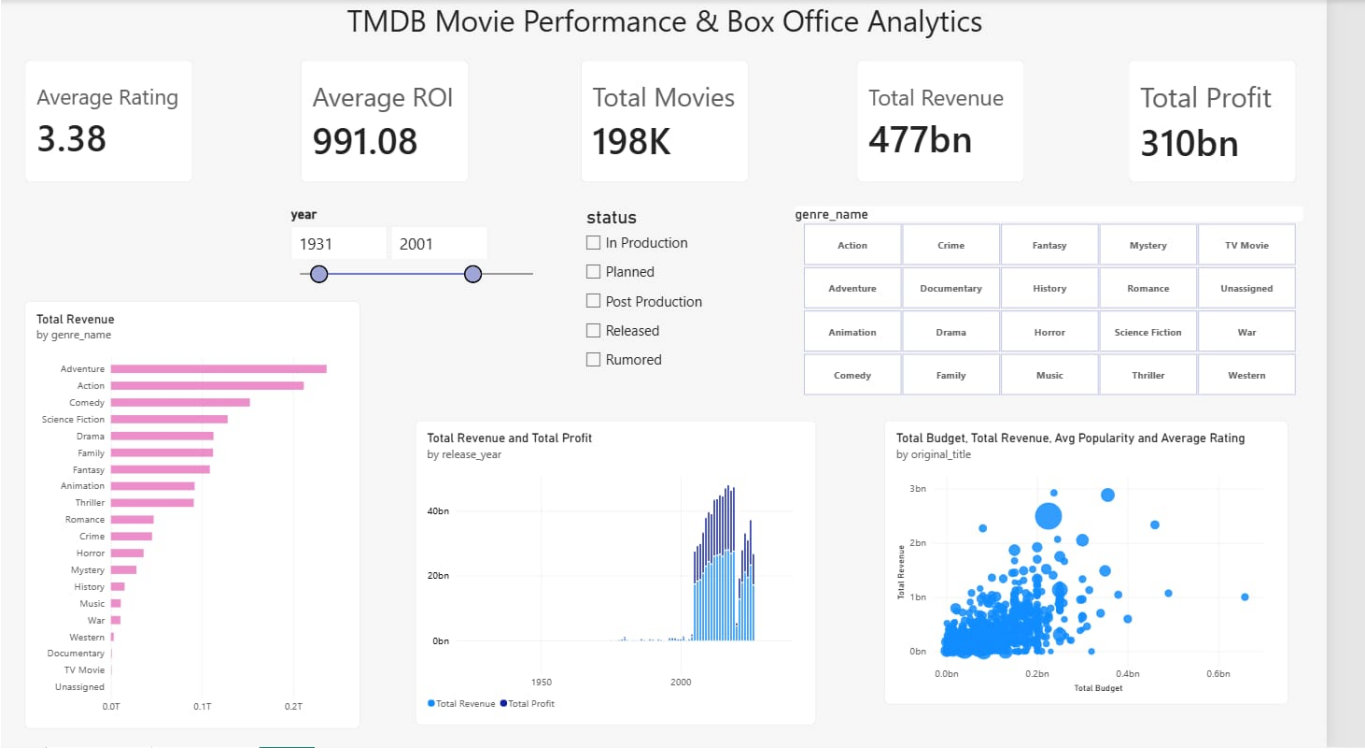


Figure 4.2: Business analytics dashboard page.

4.4 Operational / audit dashboard page

The second page, *Pipeline Execution & Data Engineering Health*, visualizes the audit table. It contains four KPI cards, one chart and one detail table:

Visual	Content as captured in Figure 4.3
KPI cards	Total Pipeline Executions 26 · Success Rate 0.85 · Successful Runs 22 · Total Rows Processed 6M
Clustered bar chart	Sum of rows_inserted and rows_updated by layer_processed. Silver-to-Gold is the only layer with a large rows_updated bar
Detail table	layer_processed, execution_end_time, execution_start_time, status, rows inserted / updated / quarantined and error_message. Table totals: 3,591,963 inserted, 2,681,276 updated, 0 quarantined

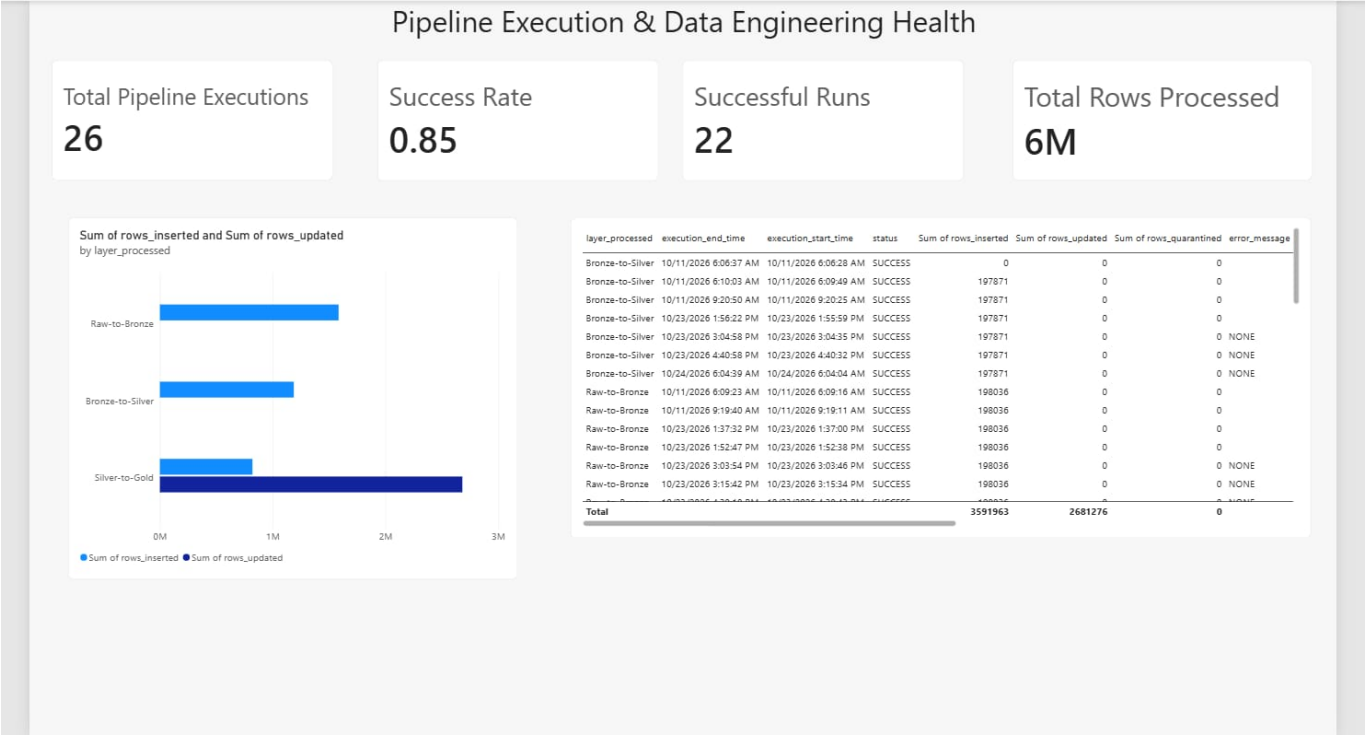


Figure 4.3: Pipeline audit and execution-health dashboard page.

Quarantine status is tracked in `quarantine_movies` (0 rows) and is queried directly in Databricks rather than loaded into the dashboard.

5. Spark UI & DAG Analysis (Gold Layer Fact Table MERGE)

5.1 Execution mechanics of the Gold-layer MERGE

The Silver-to-Gold stage executes three sequential MERGE INTO operations. Each compiles to a job with a characteristic DAG:

- **Source scan.** The Silver data is scanned (an in-memory DataFrame in the captured profile) with column pruning.
- **Deduplication.** dropDuplicates / distinct introduce Aggregate nodes with partial-to-final hash aggregation exchanges.
- **Target join.** The source is joined against the Delta target on the merge condition (for example target.movie_id = source.movie_id AND target.date_key = source.date_key) to produce matched and unmatched row sets.
- **Rewrite.** Matched rows flow to an update branch and unmatched rows to an insert branch; Delta rewrites only the affected files.
- **Commit.** The transaction commits to the Delta log (operation = MERGE) and the metrics are captured through DESCRIBE HISTORY for the audit log.

Because the fact table's clustering keys (movie_id, date_key) are also its MERGE join keys, the join predicate lines up with the physical layout, which is how the join maps to the set of touched files.

5.2 Broadcast hash joins vs. shuffle hash joins

The Gold transformation performs foreign-key lookups on movie_id and date_key. Spark's optimizer chooses between two main strategies.

- **Broadcast Hash Join.** Chosen when one side fits under spark.sql.autoBroadcastJoinThreshold (default 10 MB). The small side is broadcast to every executor and the large side is scanned once with no shuffle. `dim_date` (8,142 rows) is a textbook broadcast candidate for resolving release_date → date_key.
- **Shuffle Hash / Sort-Merge Join.** Chosen when both sides exceed the threshold; both are repartitioned by the join key. With Adaptive Query Execution (the serverless default) Spark can convert a planned sort-merge join into a shuffled

hash join or a broadcast at runtime when actual sizes turn out smaller than estimated.

In a Spark UI DAG the broadcast case shows a BroadcastExchange feeding a BroadcastHashJoin, while the shuffle case shows two Exchange nodes feeding a SortMergeJoin.

5.3 Delta log predicate pushdown and file skipping

Every Delta table carries a transaction log with per-file minimum and maximum statistics and null counts for each column. Before reading data, Delta applies data skipping:

- **Pushdown into the Delta log.** Query predicates or MERGE keys are compared with the per-file statistics, and files whose key ranges cannot match are pruned from the file list.
- **Clustering compounds this.** Because CLUSTER BY sorts rows by the clustering keys, each file's range on movie_id / date_key is narrow, so a point predicate such as movie_id = 19995 can eliminate all but the relevant file.
- **Parquet pushdown continues.** Within surviving files, row-group statistics skip further data.

This stack lets a clustered fact table support efficient keyed access as it grows.

5.4 Spark UI DAG: Gold fact table MERGE

The screenshot is the Spark UI query profile of the MERGE on `fact_movie_performance`, issued from the `gold_transformation` notebook (the query text starts with DeltaTable.forName(spark, ...), the Delta Python merge API) on Databricks Serverless compute. The graph shows execution 3 of 5 of that statement (964 ms) and the Details panel shows statement-level metrics. The start time, 2026-10-25 16:23:11 GMT+05:00 (11:23:11 UTC), falls inside the Silver-to-Gold run logged at 11:22:57 to 11:23:36 UTC (audit row 21).

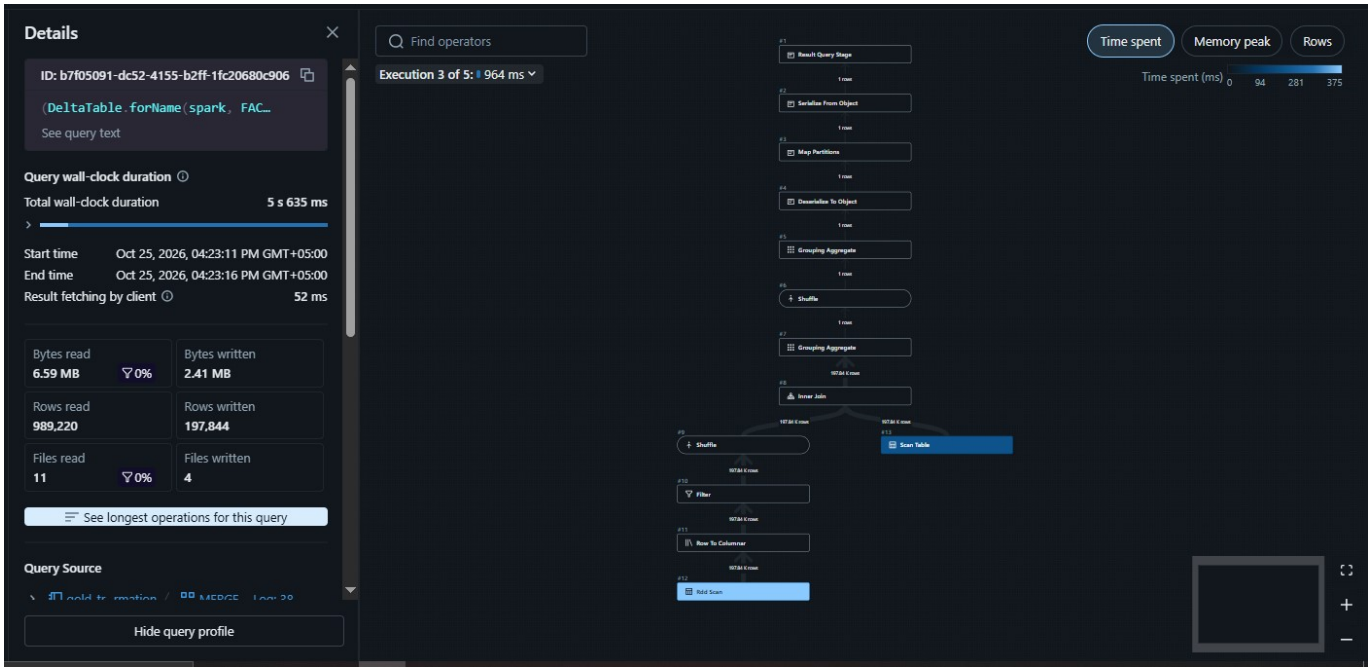


Figure 5.1: Spark UI query profile (execution 3 of 5) for the Gold fact table MERGE (fact_movie_performance), Databricks Serverless compute.

Metric (Details panel)	Value
Wall-clock duration	5.635 s
Rows read	989,220
Rows written	197,844
Bytes read / written	6.59 MB / 2.41 MB
Files read / written	11 / 4

Reading the DAG bottom-up

- **RDD Scan → Row To Columnar → Filter.** The source side is an in-memory DataFrame; rows are converted to columnar batches and filtered (about 197.84K rows).
- **Shuffle.** Source rows are redistributed ahead of the join.
- **Scan Table.** The target Delta table is scanned (about 197.84K rows).
- **Inner Join.** Source and target are joined on the MERGE condition; about 197.84K rows leave the join.
- **Grouping Aggregate → Shuffle → Grouping Aggregate.** A two-stage aggregation reduces the join output to a single row, consistent with Delta's bookkeeping of which target files are touched.
- **Serialize / Map Partitions / Deserialize → Result Query Stage.** The one-row result returns to the driver.

Key observations

- **Shuffles in the plan:** one on the source side before the join and one inside the final aggregate. For a table of about 198K rows they are cheap; the full statement completes in 5.6 s.
- **Rows read (989,220) equal exactly 5 × rows written (197,844),** consistent with the data being scanned once per each of the statement's 5 executions. This is normal MERGE behaviour (match first, then rewrite).
- **Table size:** at about 6.6 MB total across 11 files, the fact table is small enough that data skipping has little to prune; the benefit of Liquid Clustering grows with table size.
- **Most expensive operator:** time-spent colouring shows the in-memory RDD Scan as the most expensive operator; the Delta Scan Table on the target costs less.
- **Scope of the 5.6 s figure:** it covers the fact-table MERGE only, whereas the 39.5–58.5 s figures in Section 3 cover the whole Gold notebook.

6. FinOps Evaluation & Enterprise Scaling Analysis

6.1 Compute configuration

Workload	Compute	Evidence (system.billing.usage, Oct 2026)
Interactive notebook runs (master pipeline, development)	Serverless all-purpose compute	SKU `PREMIUM_ALL_PURPOSE_SERVERLESS_COMPUTE_US_EAST_OHIO`, billed on 8 days
Scheduled job tasks	Serverless jobs compute	SKU `PREMIUM_JOBS_SERVERLESS_COMPUTE_US_EAST_OHIO`; job IDs 958812722870319 (10-21) and 882712524812831 (10-25) appear in usage metadata
Power BI DirectQuery and SQL editor	Serverless SQL warehouse (2X-Small), warehouse ID a4a40a19864505c9	SKU `PREMIUM_SERVERLESS_SQL_COMPUTE_US_EAST_OHIO`, billed on 10-21, 10-23 and 10-25

Serverless means no idle all-purpose clusters billing in the background and no cluster start-up time charged to a run. Photon runs by default on serverless SQL warehouses. Recommended warehouse auto-suspend is 10 minutes.

6.2 Runtime efficiency (measured from the audit log)

Stage	Successful runs	Duration range	Mean
Raw-to-Bronze	5	8.2 s – 33.9 s	22.3 s
Bronze-to-Silver	5	18.2 s – 29.5 s	25.1 s
Silver-to-Gold	5	39.5 s – 58.5 s	46.3 s

- On 2026-10-25 the three stages ran back to back (13:19:23 to 13:22:28), a span of about 3 minutes 5 seconds including the gaps between stages.
- Gold currently re-reads all Silver rows on each run (a full MERGE of 403,857 row operations), so there is room to shorten Gold time by restricting each run to changed rows (Section 6.5).

6.3 Billed usage and list-price cost (system.billing, 2026-10-01 to 10-26)

The figures below are the DBUs recorded in system.billing.usage for this workspace, multiplied by the USD list prices in system.billing.list_prices. They are list-price estimates rather than invoice totals, and they include development and

troubleshooting work as well as pipeline runs.

SKU	Days billed	DBUs (Oct 1–26)	List price (USD/DBU)	List-price cost (USD)
Serverless all-purpose compute	8	17.362	0.75	13.02
Serverless jobs compute	13	4.483	0.35	1.57
Serverless SQL compute	3	6.380	0.70	4.47
Pipeline-relevant compute subtotal		**28.226**		**19.06**

Pipeline day	All-purpose DBU	Jobs DBU	SQL DBU	Day total at list price (USD)
2026-10-21 (first Gold run)	2.683	0.544	1.536	3.28
2026-10-23 (2 Bronze + 1 Silver run)	3.783	0.423	1.143	3.79
2026-10-25 (3 Gold + full chain, job errors)	2.095	2.033	3.701	4.87

The serverless billing tables aggregate by SKU and day, so the daily totals above cover all activity on those days. The entire billed pipeline-relevant compute for 26 days is about USD 19.06 at list price, i.e. cents per run at most.

6.4 Annual scaling view

- **Straight-line extrapolation of observed spend.** USD 19.06 over 26 days scales to about USD 268 per year (× 365 / 26). This reflects a development period, not a scheduled production cadence.
- **Active runtime of an hourly Gold refresh.** 8,760 runs/year × the measured mean of 46.3 s is about 112.6 hours of active processing per year.
- **What scales.** A MERGE's cost follows the volume of source rows read and files rewritten. While Gold re-reads all Silver rows, cost grows with catalog size; moving Gold to batch-scoped MERGEs ties it to the daily change rate instead.

6.5 Cost-optimization strategies

#	Strategy	Implementation	Expected impact
1	Warehouse auto-suspension (10 min)	Keep auto-suspend at 10 minutes on the 2X-Small warehouse so DirectQuery or SQL usage does not leave it running.	Removes idle SQL-warehouse billing between BI sessions.
2	Off-peak OPTIMIZE + VACUUM	Run OPTIMIZE on fact_movie_performance and dim_movies as a separate nightly job, followed by VACUUM with a 168-hour retention, instead of inside each Gold run.	Shortens each Gold run; keeps files compact; VACUUM reclaims storage left by Bronze and Gold rewrites.
3	Incremental ingestion	Bronze appended a full 198,036-row snapshot on each of 4 runs (989,664 rows, 212 MB). Pull only changed IDs for daily runs and keep full loads for backfills.	Cuts Bronze growth and Silver work to the daily change rate; the 197,871-row Silver stays the same.
4	Batch-scoped Gold MERGE	Gold re-processes all 197,871 Silver rows on each run (403,857 row operations). Restrict the MERGE source to rows changed since the last Gold run.	Gold time and DBUs fall in proportion to the daily change rate.
5	Auto-scaling serverless sizing	Size up for backfills and down for incremental runs as volume grows.	Pay for actual load, with no re-architecture.

Additional guardrails

- **Per-layer DBU attribution.** Tag each stage's compute so system.billing.usage can attribute DBUs to Raw-to-Bronze, Silver and Gold.
- **Budget alerts.** Alert on anomalous daily DBU spend (for example more than 3 times the trailing 7-day average). On 2026-10-25 jobs compute reached 2.03 DBUs against 0.12 to 0.54 on other job days, which is the kind of spike such an alert would catch.
- **Job health alerting.** Add failure notifications on the Databricks Job so job state and the audit table stay in step.
- **BI query cost.** DirectQuery runs a warehouse query on each interaction. For a busier deployment, Import mode with a refresh scheduled right after the pipeline would keep the warehouse awake only for the refresh.

7. Verification Basis

The figures in this report were gathered as follows.

Verified by live query on 2026-10-26

- All 25 audit-log rows, per-layer summaries, and distinct load_type and parameter values.
- Row counts of every table in `workspace.tmdb`; Silver and fact schemas (24 and 11 columns).
- Bronze batch composition (5 batches, 989,664 rows) and the number of distinct movie IDs (197,871).
- Distinct and range values of pulled_at in Silver and Bronze (2 values, 2026-10-01 to 2026-10-08).
- Billing usage and list prices for the workspace, and the job run timeline.
- Bronze DESCRIBE DETAIL (7 files, 212,440,218 bytes, no partitioning or clustering).
- Figures 1.1, 4.1 to 4.3 and 5.1 are screenshots supplied by the author; the values quoted from them are read directly from the images.

8. Deliverables Map

Deliverable	Location
GitHub repository (PySpark notebooks)	TMDB-Data-Engineering-Project (contents below)
Power BI file	Submitted as a .pbix file with this report
Final report	This document (PDF)

Repository contents: `00_master_pipeline`, `bronze/bronze_injestion`, `silver/silver_injestion`, `gold/gold_transformation`, `log/00_audit_logging`, `images/` (report figures), `staging/` sample payloads and `data_retrieval_scripts/` (TMDB extraction).

Conclusion

The TMDB Movie Analytics Lakehouse delivers a working medallion pipeline with an auditable record of its own behaviour. It combines immutable Bronze replayability (989,664 raw rows across 5 batches), PERMISSIVE parsing with quarantine containment, re-runnable MERGE loading in which Silver and Gold counts held steady across repeated runs, a liquid-clustered Gold star schema (197,871 movies, 8,142 dates and 197,844 facts, reconciling to Silver minus 27 un-datable rows), a Power BI model with business and pipeline-audit pages, and complete audit coverage including 10 trapped development failures. Gold refreshes took 39.5 to 58.5 seconds, and the compute billed for the whole month to date is about USD 19 at list price. The main opportunities for the next iteration are incremental ingestion, batch-scoped Gold loads and job-level alerting.